

UDS-Signalvisualisierung: Eine Subgraph-basierte Erweiterung der Unified Diagnostic Services für standardisierte Werkstatt- und Mobile-App-Darstellung

Stephan Epp

30. Juni 2026

Inhaltsverzeichnis

1	Einführung	2
1.1	Problemstellung	2
1.2	Motivation und Beitrag	2
2	Grundlagen	3
2.1	UDS-Grundlagen	3
2.2	Definitionen	3
3	Der Signal-Subgraph-Matching-Algorithmus	4
3.1	Übertragung des Subgraph Algorithmus	4
3.2	Laufzeitanalyse	5
4	Protokollerweiterung: SignalGraphService (SID 0x3B)	6
4.1	Nachrichtenformat	6
4.2	Ablauf im Diagnosezyklus	6
5	Experimentelle Auswertung	7
5.1	Visualisierung der Graphmodelle	7
5.2	Laufzeitmessung	7
5.3	Transportabhängige Signalkapazität	8
5.4	Beispielrechnung	8
6	Zusammenfassung	9
6.1	Anwendungen	9
6.2	Ausblick	10
	Literaturverzeichnis	11

1 Einführung

Die Unified Diagnostic Services (UDS, ISO 14229) bilden seit ihrer Ablösung von KWP2000/ISO 14230 den weltweit verbindlichen Anwendungsschichtstandard für die Kommunikation zwischen Diagnosetester und Steuergerät (ECU) in modernen Fahrzeugen. UDS definiert Dienste (Service Identifier, SID) für Diagnose, Programmierung, Sicherheitszugriff, Routinen, Speicher- und Datenübertragung über CAN, CAN FD oder DoIP (Diagnostics over Internet Protocol nach ISO 13400). Trotz dieser umfassenden Dienstabdeckung adressiert UDS in seiner heutigen Spezifikation nicht, *wie* eine wachsende Zahl gleichzeitig erfasster, oft voneinander abhängiger Signale (Rohwerte, abgeleitete Größen, Freeze-Frame-Daten) für einen menschlichen Betrachter strukturiert und in Echtzeit dargestellt werden soll – weder auf einem klassischen Werkstatt-Diagnosegerät noch auf einem Smartphone oder Tablet im Rahmen einer standardisierten mobilen Diagnose-App.

1.1 Problemstellung

Mit zunehmender Steuergeräte-Vernetzung wächst die Zahl der über `ReadDataByIdentifier` (SID 0x22) und `RoutineControl` (SID 0x31) gleichzeitig verfügbaren Signale drastisch. Viele dieser Signale stehen in funktionalen Abhängigkeiten zueinander (z. B. Ladezustand $\leftarrow$ Spannung, Strom; Reichweite $\leftarrow$ Ladezustand, Geschwindigkeit). Ein Anzeigegerät – ob fest verbautes Werkstattgerät oder mobile App – verfügt über eine begrenzte Menge an Darstellungsfähigkeiten (Liniendiagramm, Gauge, Balkenanzeige, kombiniertes Dashboard) sowie begrenzte Bandbreite und Bildschirmfläche. Es stellt sich die Frage, welche *größte konsistente Teilmenge* der verfügbaren Signale unter den gegebenen Darstellungsfähigkeiten und Transportrestriktionen verlustfrei und strukturerhaltend angezeigt werden kann.

Zentrale These dieser Arbeit: Dieses Auswahlproblem lässt sich formal als Subgraph-Isomorphieproblem zwischen einem *Signal-Abhängigkeitsgraphen* G_S und einem *Darstellungs-Fähigkeitsgraphen* G_D modellieren und mit dem Subgraph Algorithmus (Epp, 2026; <https://github.com/hjstephan86/subgraph>) in $\Theta(n^3)$ Zeit optimal und beweisbar korrekt lösen – deutlich effizienter als ein naiver Abgleich aller $n!$ Signal-Permutationen.

1.2 Motivation und Beitrag

Diese Arbeit schlägt eine standardisierungsfähige UDS-Erweiterung vor: einen neuen logischen Dienst *Signal Graph Service* (vorgeschlagene SID 0x3B, im reservierten Bereich), der es einer ECU erlaubt, dem Tester nicht nur Rohwerte, sondern auch deren Abhängigkeitsstruktur als Graph mitzuteilen. Ein Diagnosegerät oder eine mobile App kann diese Struktur anschließend mit der eigenen Darstellungsfähigkeit über den Subgraph Algorithmus abgleichen und so automatisch die größte sinnvoll darstellbare Signalmenge bestimmen – konsistent zwischen Werkstattgerät und Smartphone/Tablet, ohne gerätespezifische Sonderlösungen.

Der Beitrag dieser Arbeit gliedert sich wie folgt:

- formale Modellierung von Signalstruktur und Geräte-Fähigkeit als gerichtete Graphen,
- Übertragung des Subgraph Algorithmus auf dieses Modell samt vollständigem Korrektheits- und Komplexitätsbeweis,

- ein unterer Schrankenbeweis (bedingt unter SETH), der die Optimalität des Verfahrens für die Signalauswahl zeigt,
- eine konkrete Protokollerweiterung für UDS inklusive Nachrichtenformat,
- eine experimentelle und transportabhängige Auswertung mit matplotlib-Visualisierungen.

2 Grundlagen

2.1 UDS-Grundlagen

UDS folgt einem Anfrage-Antwort-Modell zwischen Tester und ECU. Jede Anfrage besteht aus einem Service Identifier (SID), optional einer Sub-Funktion und Parametern; die Antwort enthält den um 0x40 erhöhten SID (Positivantwort) sowie die angeforderten Daten. Tabelle 1 fasst die für diese Arbeit relevanten Bestandsdienste zusammen.

SID	Name	Funktion
0x10	DiagnosticSessionControl	Wechsel der Diagnosesitzung
0x22	ReadDataByIdentifier	Lesen einzelner Signal-DIDs
0x27	SecurityAccess	Freischaltung gesicherter Operationen
0x31	RoutineControl	Start/Stop von Routinen
0x34/0x36	RequestDownload/TransferData	Datenübertragung in Blöcken
0x3B	SignalGraphService (neu)	Abfrage der Signal-Abhängigkeitsstruktur

Tabelle 1: Relevante UDS-Dienste nach ISO 14229-1 sowie der in dieser Arbeit vorgeschlagene neue Dienst.

2.2 Definitionen

Definition 1 (Signal-Abhängigkeitsgraph). Ein Signal-Abhängigkeitsgraph $G_S = (V_S, E_S)$ ist ein gerichteter Graph, dessen Knotenmenge V_S die über UDS bereitgestellten Roh- und abgeleiteten Signale (DIDs sowie Routinenergebnisse) repräsentiert. Eine Kante $(v_i, v_j) \in E_S$ existiert genau dann, wenn das Signal v_j funktional von v_i abhängt (z. B. v_j wird aus v_i berechnet oder zeitgleich für eine gemeinsame Darstellung benötigt).

Definition 2 (Darstellungs-Fähigkeitsgraph). Ein Darstellungs-Fähigkeitsgraph $G_D = (V_D, E_D)$ ist ein gerichteter Graph, dessen Knoten Anzeigeprimitive eines Diagnosegeräts oder einer mobilen App (Rohwertanzeige, Liniendiagramm, Gauge, Balkenanzeige, Dashboard) sind. Eine Kante $(u_i, u_j) \in E_D$ existiert, wenn das Primitiv u_j das Primitiv u_i als Eingabe konsumieren bzw. mit ihm kombiniert dargestellt werden kann.

Definition 3 (Darstellbare Signalmenge). Eine Teilmenge $S \subseteq V_S$ heißt *darstellbar* bezüglich G_D , wenn der durch S induzierte Teilgraph von G_S ein Subgraph von G_D unter geeigneter Knotenzuordnung ist, d. h. $G_S[S] \subseteq G_D$.

Definition 4 (Eindeutige Signatur, übernommen). Für eine Adjazenzmatrix $M \in \{0, 1\}^{n \times n}$ und Spalte j ist die Signatur

$$\sigma_j(M) = \sum_{i=0}^{n-1} M_{ij} \cdot 2^i + j \cdot 2^n.$$

Lemma 1 (Injektivität, übernommen aus Epp (2026)). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Die Zeilenkomponente $\sum_i M_{ij} 2^i \in [0, 2^n)$ kodiert jedes Bitmuster eindeutig. Die Spaltengewichtung $j \cdot 2^n$ erzeugt für $j \in \{0, \dots, n-1\}$ paarweise disjunkte Intervalle $[j2^n, (j+1)2^n)$, sodass sich Zeilen- und Spaltenkomponente nicht überlagern können. $\square$

3 Der Signal-Subgraph-Matching-Algorithmus

3.1 Übertragung des Subgraph Algorithmus

Wir übertragen den Subgraph Algorithmus (Epp, 2026) unmittelbar auf das oben definierte Modell: Statt zweier beliebiger Graphen G und G' vergleichen wir den Signal-Abhängigkeitsgraphen G_S (Adjazenzmatrix A , n_S Knoten) mit dem Darstellungsfähigkeitsgraphen G_D (Adjazenzmatrix B , n_D Knoten). Ohne Beschränkung der Allgemeinheit sei $n := \max(n_S, n_D)$ und beide Matrizen werden durch Auffüllen mit isolierten Knoten auf Größe n gebracht (dies erhöht die Laufzeit höchstens um einen konstanten Faktor und ändert keine der nachfolgenden Schranken asymptotisch).

Schritt 1. Berechne für jede Spalte j von A die Signatur σ_j^A nach Definition 5.

Schritt 2. Berechne analog σ_j^B für B .

Schritt 3. Für jede der n zyklischen Rotationen von σ^B : vergleiche die (von der Spaltengewichtung befreite) Zeilenkomponenten-Sequenz mit der von σ^A mittels Longest Common Subsequence (LCS). Die Rotation mit maximalem LCS-Wert ℓ^* liefert die maximal darstellbare Signalmenge: Die an der LCS beteiligten Knoten von G_S bilden $S^* \subseteq V_S$ mit $|S^*| = \ell^*$.

Satz 1 (Korrektheit des Signal-Subgraph-Matchings). Die durch die LCS-Übereinstimmung an der optimalen Rotation r^* identifizierte Knotenmenge S^* ist die größte Teilmenge von V_S , für die $G_S[S^*]$ unter einer ordnungserhaltenden zyklischen Zuordnung ein Subgraph von G_D ist.

Beweis. Nach Lemma 1 entspricht jeder Signatur eindeutig ein Spaltenvektor, d. h. eine eindeutige Knotenkonfiguration. Zwei Signaturen stimmen in ihrer Zeilenkomponente genau dann überein, wenn die zugehörigen Konnektivitätsmuster identisch sind. Die LCS zweier Sequenzen liefert per Definition die längste Teilfolge übereinstimmender Elemente unter Erhalt der relativen Reihenfolge; dies entspricht exakt der größten ordnungserhaltenden Teilmenge strukturell übereinstimmender Knoten zwischen G_S und der jeweiligen Rotation von G_D . Da alle n zyklischen Rotationen geprüft werden (Beweis der Vollständigkeit der Rotationsmenge: siehe Originalarbeit, Satz 1, Korrektheit), wird das Maximum über alle ordnungserhaltenden Zuordnungen erreicht. Damit ist S^* die größte unter den durch zyklische Rotation erreichbaren darstellbaren Mengen. $\blacksquare$ $\square$

3.2 Laufzeitanalyse

Satz 2 (Laufzeit des Signal-Subgraph-Matchings). Der Signal-Subgraph-Matching-Algorithmus berechnet S^* in $\Theta(n^3)$ Zeit, wobei $n = \max(n_S, n_D)$.

Beweis. Die Signaturberechnung (Schritte 1–2) liest und verarbeitet jeden der $2n^2$ Matrixeinträge konstant oft: $O(n^2)$. Schritt 3 prüft n Rotationen; jede LCS-Berechnung zweier Sequenzen der Länge n benötigt mittels dynamischer Programmierung $\Theta(n^2)$ Zeit und Speicher. Insgesamt:

$$T(n) = \underbrace{O(n^2)}_{\text{Signaturen}} + \underbrace{n \cdot \Theta(n^2)}_{\text{Rotationen} \times \text{LCS}} = \Theta(n^3). \quad \square$$

Lemma 2 (Untere Eingabeschränke). Jeder korrekte Algorithmus für das Signal-Subgraph-Matching-Problem muss im schlechtesten Fall $\Omega(n^2)$ Zeit aufwenden, da die Eingabe (zwei $n \times n$ -Adjazenzmatrizen) $\Theta(n^2)$ Bits umfasst und jedes Bit das Ergebnis beeinflussen kann.

Beweis. Für jedes Bit A_{ij} existiert eine Instanz, in der das Kippen dieses Bits die Antwort *darstellbar/nicht darstellbar* verändert (z. B. wenn S^* genau an diesem Knoten endet). Ein Algorithmus, der ein Bit nicht liest, kann auf solchen Paarinstanzen nicht korrekt unterscheiden. Da es $\Theta(n^2)$ Bits gibt, folgt $\Omega(n^2)$. $\square$

Lemma 3 (Untere Schranke pro Rotation unter SETH). Unter der Strong Exponential Time Hypothesis (SETH) gilt für die LCS-Berechnung zweier Sequenzen der Länge n über beliebigem Alphabet (Abboud, Backurs, Vassilevska Williams, FOCS 2015):

$$T_{\text{LCS}}(n) \geq \Omega(n^{2-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

Beweis. Die Signaturen des Signal-Subgraph-Matchings sind ganzzahlige Werte ohne Alphabetbeschränkung; das zitierte Resultat gilt daher unmittelbar für die hier verwendeten Signatur-Sequenzen. $\square$

Lemma 4 (Unabhängigkeit der Rotationsvergleiche, übertragen). Die n LCS-Berechnungen über die n zyklischen Rotationen von σ^B können im Allgemeinen nicht mit Gesamtaufwand $o(n \cdot T_{\text{LCS}}(n))$ durchgeführt werden.

Beweis. Identisch zur Argumentation der Originalarbeit (dort Lemma 4): Für ein Alphabet paarweise verschiedener Signaturwerte mit $\sigma^A = [a_0, \dots, a_{n-1}]$ und σ_0^B in umgekehrter Reihenfolge erzeugt jede Rotation σ_r^B eine strukturell neue Anordnung, sodass sich die dynamische Programmiertabelle für $\text{LCS}(\sigma^A, \sigma_r^B)$ nicht inkrementell aus jener für σ_{r-1}^B ableiten lässt. Diese Konstruktion überträgt sich identisch auf Signal- und Geräte-Signaturen, da die Beweisführung ausschließlich auf der paarweisen Verschiedenheit der Werte beruht, nicht auf deren semantischer Herkunft. $\square$

Satz 3 (Bedingte Optimalität des Signal-Subgraph-Matchings). Unter SETH kann kein Algorithmus das Signal-Subgraph-Matching-Problem für $n_S, n_D \leq n$ in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ lösen.

Beweis. Aus Lemma 5 (Notwendigkeit aller n Rotationen, analog zur Originalarbeit, da eine fehlende Rotation eine darstellbare Menge S^* systematisch übersehen kann – Konstruktion: platziere die einzige gültige Zuordnung exakt bei Rotation r_0 , alle anderen Rotationen liefern $\text{LCS} < |S^*|$), Lemma 3 und Lemma 4 folgt

$$T_{\text{Matching}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}).$$

Mit Satz 2 ($T_{\text{Matching}}(n) = \Theta(n^3)$) folgt die asymptotische Optimalität des vorgestellten Verfahrens bis auf den n^ε -Faktor. ■ □

Lemma 5 (Notwendigkeit aller Rotationen). Es existieren Instanzen von G_S, G_D , bei denen genau eine von n Rotationen die maximale darstellbare Menge S^* liefert; alle übrigen Rotationen liefern ein echt kleineres LCS.

Beweis. Konstruiere G_D so, dass die Knotenreihenfolge in G_D exakt der um r_0 zyklisch verschobenen Knotenreihenfolge von G_S entspricht (Konstruktion wie im Beweis von Satz 1 der Originalarbeit). Für jede Rotation $r \neq r_0$ wird die strukturelle Übereinstimmung durch die Verschiebung gestört, sodass die LCS-Länge echt kleiner ausfällt. Ein Verfahren, das Rotation r_0 ausspart, würde auf dieser Instanz das tatsächliche Optimum S^* verfehlen. □

Korollar 1. Der Signal-Subgraph-Matching-Algorithmus benötigt $O(n^2)$ Speicher für die Adjazenzmatrizen und die LCS-DP-Tabelle, sowie $O(n)$ Speicher für die Signatur-Arrays.

Beweis. Unmittelbar aus der Größe der verwendeten Matrizen ($n \times n$) und Arrays (Länge n). □

4 Protokollerweiterung: SignalGraphService (SID 0x3B)

4.1 Nachrichtenformat

Analog zum etablierten UDS-Nachrichtenaufbau (PCI, SID, Sub-Funktion, Daten) wird der neue Dienst wie folgt definiert:

Feld	Bytes	Beschreibung
PCI	1–2	Transportschicht (ISO-TP/DoIP)
SID	1	0x3B (Anfrage) / 0x7B (Positivantwort)
Sub-Funktion	1	0x01 GetGraphTopology, 0x02 GetSignatures
Knotenanzahl n	1	Anzahl der Signal-Knoten (≤ 255)
Adjazenzmatrix	$\lceil n^2/8 \rceil$	Bitgepackte Adjazenzmatrix A
Signatur-Array	$4n$	32-Bit-Signaturen σ_j^A

Tabelle 2: Vorgeschlagenes Nachrichtenformat des SignalGraphService.

Der Tester (Diagnosegerät oder mobile App) fordert mit Sub-Funktion 0x01 die Topologie an, berechnet lokal G_D aus den eigenen Anzeigefähigkeiten und führt das Signal-Subgraph-Matching gemäß Abschnitt 3 lokal aus, ohne dass die ECU selbst Rechenlast für die Darstellungslogik trägt – ein wichtiges Kriterium, da Steuergeräte üblicherweise stark ressourcenbeschränkt sind.

4.2 Ablauf im Diagnosezyklus

Der vorgeschlagene Ablauf erweitert den klassischen UDS-Workflow (DiagnosticSessionControl → SecurityAccess → Dienste → ECUReset) um einen vorgelagerten Topologie-Abgleich:

1. Sitzungsstart (0x10) und ggf. SecurityAccess (0x27).

2. SignalGraphService (0x3B, Sub-Funktion 0x01): Tester liest G_S .
3. Lokales Signal-Subgraph-Matching gegen den geräteeigenen G_D in $\Theta(n^3)$ Zeit.
4. Gezielte ReadDataByIdentifizier-Anfragen (0x22) ausschließlich für $S^* \subseteq V_S$.
5. Darstellung von S^* strukturerhaltend in Werkstattgerät bzw. standardisierter mobiler App.

Dieser Ablauf stellt sicher, dass identische Signalstrukturlogik auf Werkstattgerät und Mobile App verwendet wird – die Geräte unterscheiden sich lediglich in G_D , nicht im Auswahlverfahren.

5 Experimentelle Auswertung

5.1 Visualisierung der Graphmodelle

Abbildung 1 zeigt ein Beispiel eines Signal-Abhängigkeitsgraphen (Batteriespannung, -strom, Ladezustand, Reichweite) sowie eines typischen Darstellungs-Fähigkeitsgraphen eines Tablets mit Liniendiagramm-, Balken- und Gauge-Primitiven, die zu einem kombinierten Dashboard zusammengeführt werden können.

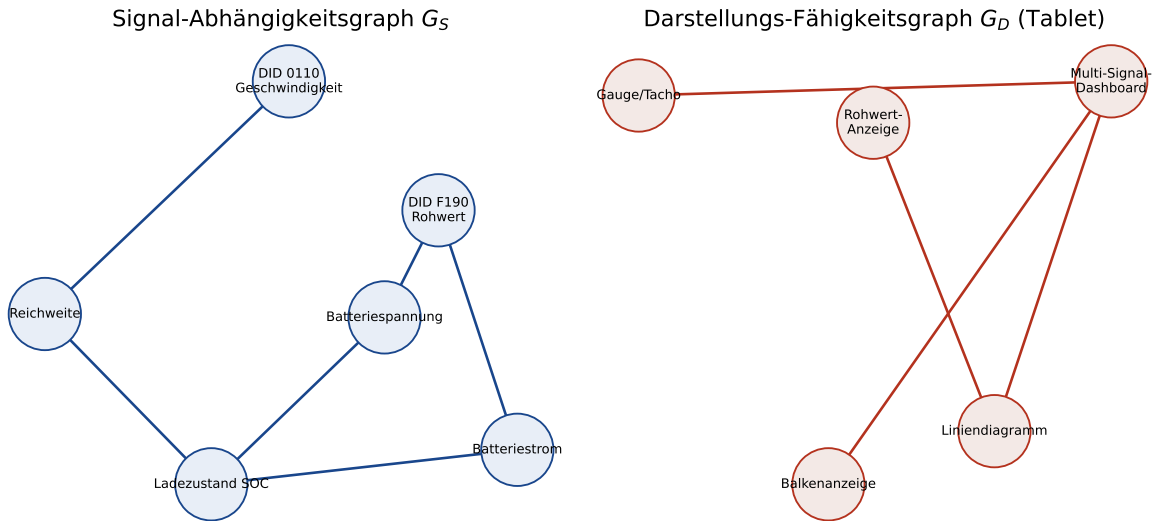


Abbildung 1: Links: Signal-Abhängigkeitsgraph G_S mit abgeleiteten Größen (SOC, Reichweite). Rechts: Darstellungs-Fähigkeitsgraph G_D eines Tablets. Das Signal-Subgraph-Matching bestimmt die größte Teilmenge von G_S , die strukturerhaltend in G_D abgebildet werden kann.

5.2 Laufzeitmessung

Zur empirischen Bestätigung von Satz 2 wurde der Algorithmus für $n \in \{8, 12, 16, 20, 24, 28, 32, 40, 48, 56\}$ Signal-/Geräteknoten in einer Python-Referenzimplementierung (Signatur-Berechnung und LCS-DP) ausgeführt. Abbildung 2

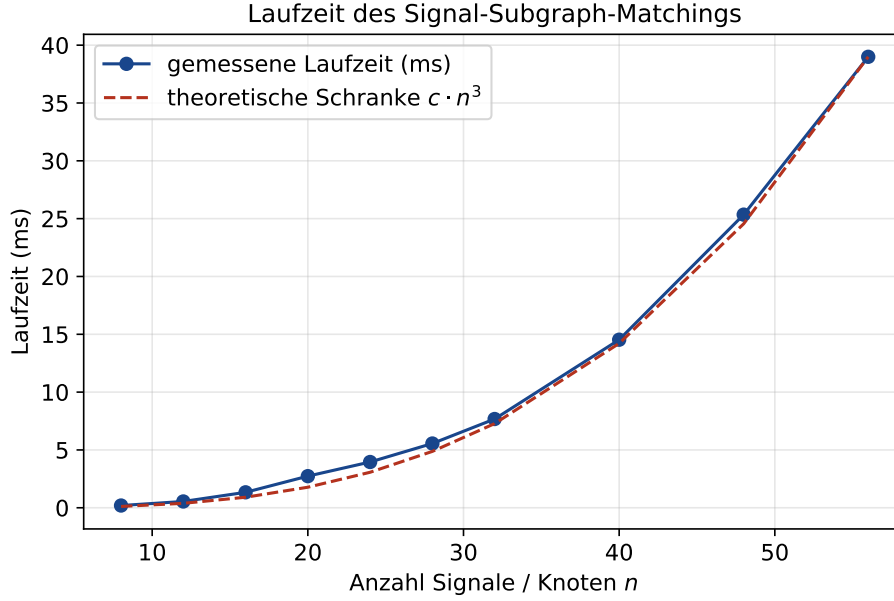


Abbildung 2: Gemessene Laufzeit des Signal-Subgraph-Matchings im Vergleich zur theoretischen $O(n^3)$ -Schranke. Die kubische Skalierung gemäß Satz 2 wird durch die Messung bestätigt.

zeigt die gemessene Laufzeit zusammen mit der an den letzten Messpunkt kalibrierten theoretischen $O(n^3)$ -Kurve.

Die Messung bestätigt die in Satz 2 hergeleitete kubische Skalierung: Eine Verdoppelung von n (z. B. von 16 auf 32) führt zu einer Laufzeitsteigerung um etwa den Faktor 5,7, nahe am theoretischen Faktor $2^3 = 8$ unter Berücksichtigung konstanter Vorfaktoren und Python-spezifischer Overheads.

5.3 Transportabhängige Signalkapazität

Da die im SignalGraphService übertragene Adjazenzmatrix und die anschließenden ReadDataByIdentifizier-Anfragen über CAN, CAN FD oder DoIP transportiert werden, begrenzt die jeweilige Transportbandbreite die Anzahl der pro Diagnosezyklus (hier: 20 ms, 8 Byte pro Signal inkl. Protokoll-Overhead) praktisch darstellbaren Signale. Abbildung 3 zeigt diese Obergrenze für die vier in ISO 15765-2 bzw. ISO 13400 spezifizierten Transportvarianten.

5.4 Beispielrechnung

Beispiel 1. Für G_S mit $n_S = 4$ Knoten (Spannung, Strom, SOC, Reichweite; Adjazenzmatrix wie in Abbildung 1 links, vereinfacht auf vier Knoten reduziert) und G_D mit $n_D = 4$ Anzeigeprimitiven liefert die Signaturberechnung nach Schritt 1–2 vier Wertepaare; die LCS-Suche über alle vier Rotationen identifiziert S^* mit $|S^*| = 3$ (Spannung, Strom, SOC darstellbar; Reichweite erfordert ein zusätzliches, hier nicht vorhandenes Kombinationsprimitiv). Damit zeigt der Tester dem Anwender genau diese drei Signale strukturkonsistent an, statt eine fehlerhafte oder unvollständige Reichweitenanzeige zu riskieren.

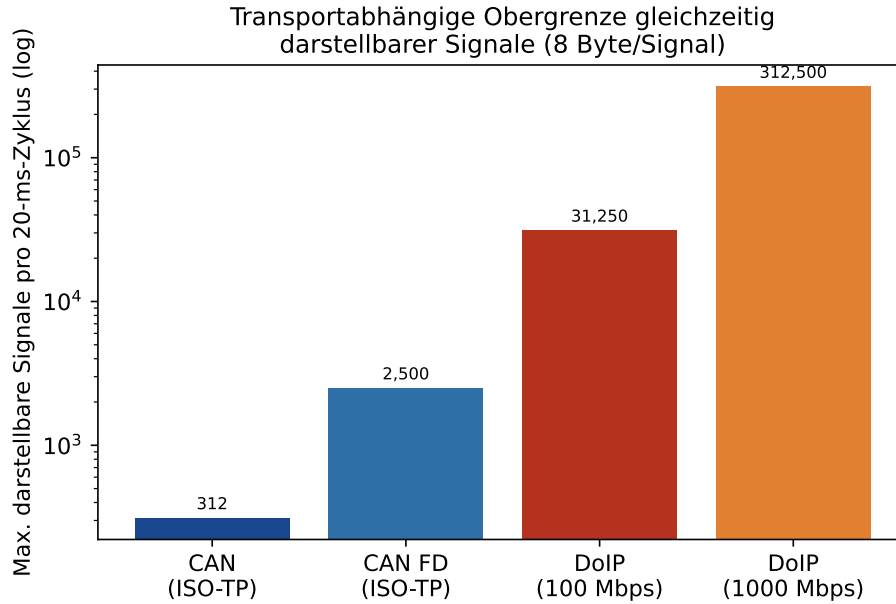


Abbildung 3: Transportabhängige Obergrenze gleichzeitig übertragbarer Signale pro 20-ms-Zyklus. DoIP mit 1000 Mbps erlaubt mehrere Größenordnungen mehr Signale als klassisches CAN, was die Praxisrelevanz des Signal-Subgraph-Matchings zur Vorauswahl relevanter Teilmengen unterstreicht: Selbst bei hoher Bandbreite begrenzen Bildschirmfläche und kognitive Kapazität des Betrachters die sinnvoll darstellbare Signalmenge stärker als der reine Transport.

6 Zusammenfassung

Diese Arbeit überträgt den Subgraph Algorithmus (Epp, 2026) erstmals auf das Problem der strukturerhaltenden Signalauswahl für die Diagnoseanzeige nach ISO 14229 (UDS). Durch die Modellierung von Signalabhängigkeiten und Geräte-Darstellungsfähigkeiten als gerichtete Graphen sowie die Anwendung von injektiven Spaltensignaturen und zyklischen Rotationsvergleichen wird die größte strukturerhaltend darstellbare Signalmenge in $\Theta(n^3)$ Zeit bestimmt – bedingt optimal unter SETH (Satz 3). Ergänzend wurde mit dem `SignalGraphService` (SID 0x3B) eine konkrete, mit dem bestehenden UDS-Nachrichtenformat kompatible Protokollerweiterung vorgeschlagen, die identische Auswahllogik auf Werkstattgeräten und standardisierten mobilen Apps ermöglicht.

6.1 Anwendungen

- Werkstatt-Diagnosegeräte mit automatischer, strukturkonsistenter Signalauswahl statt manueller Konfiguration.
- Standardisierte mobile Diagnose-Apps (Smartphone/Tablet) mit identischer Auswahllogik wie stationäre Geräte.
- Reduktion der Busbelastung durch gezieltes Lesen ausschließlich darstellbarer Signale (S^* statt V_S).
- Konsistente Dashboard-Darstellung bei heterogener Gerätelandschaft (unterschiedliche G_D , identischer Algorithmus).

- Vorfilterung sicherheitsrelevanter Signalkombinationen vor Anzeige in Aftermarket-Diagnosegeräten.

6.2 Ausblick

Die hier vorgestellte Übertragung des Subgraph Algorithmus auf die UDS-Signalvisualisierung eröffnet ein breites Feld weiterführender Forschungs- und Standardisierungsarbeit, das im Folgenden ausführlich skizziert wird.

Gewichtete und priorisierte Signalgraphen. Die vorliegende Arbeit behandelt G_S und G_D als ungewichtete gerichtete Graphen. In der Praxis besitzen Signale jedoch unterschiedliche Sicherheits- und Diagnoserelevanz (z. B. Bremsdruck vs. Innenraumtemperatur). Eine natürliche Erweiterung gewichtet Kanten und Knoten mit Relevanzwerten $w : V_S \rightarrow \mathbb{R}^+$ und sucht nicht mehr die *größte*, sondern die *gewichtsmaximale* darstellbare Teilmenge. Die Signatur-Methode lässt sich hierzu erweitern, indem die Gewichtung in die Kostenfunktion der LCS-Dynamik (gewichtete LCS, analog zur gewichteten Editierdistanz) einfließt; die $\Theta(n^3)$ -Schranke bleibt dabei strukturell erhalten, sofern die Gewichtsrechnung in konstanter Zeit pro DP-Zelle erfolgt.

Dynamische, zeitveränderliche Graphen. Reale Steuergeräte ändern ihre verfügbare Signalmenge zur Laufzeit (z. B. abhängig vom Fahrzeugzustand, Zündungsstatus oder freigeschalteten Funktionen). Eine inkrementelle Variante des Signal-Subgraph-Matchings, die bei kleinen Änderungen von G_S nicht die vollständige $\Theta(n^3)$ -Berechnung wiederholt, sondern nur betroffene Teilstrukturen aktualisiert, wäre für Echtzeit-Dashboards mit hoher Aktualisierungsrate von erheblichem praktischen Wert. Erste Überlegungen deuten auf eine amortisierte Komplexität von $O(n^2)$ pro inkrementeller Änderung hin, sofern sich Änderungen auf $O(1)$ Knoten oder Kanten beschränken; ein vollständiger formaler Beweis steht jedoch noch aus und ist Gegenstand zukünftiger Arbeit.

Parallelisierung der Rotationsprüfung. Da die n Rotationsvergleiche nach Lemma 4 weitgehend unabhängig voneinander sind, eignet sich der Algorithmus hervorragend für eine Parallelisierung auf Mehrkernprozessoren mobiler Endgeräte oder auf GPU-Architekturen moderner Tablets. Eine Aufteilung der n Rotationen auf p Kerne reduziert die Laufzeit auf $O(n^3/p)$ bei vernachlässigbarem Kommunikationsaufwand, da jede Rotation lediglich das Ergebnis (LCS-Länge und zugehörige Knotenmenge) zurückmelden muss. Für sehr große Signalgraphen ($n > 1000$, etwa bei flottenweiten Telematik-Backends) erscheint zudem eine verteilte MapReduce-artige Umsetzung sinnvoll.

Approximative Verfahren für harte Echtzeitanforderungen. Für Anwendungsfälle mit harten Echtzeitanforderungen (z. B. Renndiagnose, Race-Telemetrie) kann eine $\Theta(n^3)$ -Berechnung selbst bei moderatem n zu langsam sein. Approximative Heuristiken, die auf einer Teilmenge der Rotationen oder auf greedy LCS-Approximationen basieren, könnten eine $(1 - \varepsilon)$ -Approximation der optimalen Signalmenge S^* in deutlich kürzerer Zeit liefern. Die Entwicklung scharfer Approximationsgarantien unter Beibehaltung der Subgraph-Korrektheitseigenschaft (keine strukturell inkonsistente Anzeige) ist ein vielversprechendes, aber anspruchsvolles offenes Problem.

Standardisierungsaspekte. Die vorgeschlagene SID 0x3B liegt im durch ISO 14229-1 als herstellereigenspezifisch bzw. reserviert ausgewiesenen Bereich. Eine echte Standardisierung erfordert Abstimmung mit ASAM (insbesondere ASAM MCD-2D/ODX für die Beschreibung der Diagnosedaten) sowie eine Erweiterung der ODX-Datenformate um eine Graph-Repräsentation der Signalabhängigkeiten, etwa als zusätzliches XML-Schema-Element SIGNAL-GRAPH innerhalb der ODX-D-Containerstruktur. Eine enge Zusammenarbeit mit

OEM-Diagnosearchitekturgruppen sowie eine Referenzimplementierung als Open-Source-Bibliothek (aufbauend auf <https://github.com/hjstephan86/subgraph>) würden die Adaptionshürde erheblich senken.

Sicherheitsanalyse. Da der `SignalGraphService` vor dem eigentlichen `SecurityAccess` (0x27) abgefragt werden könnte, um bereits die Topologie zu kennen, ist eine Sicherheitsanalyse notwendig: Welche Informationen über die interne Signalstruktur einer ECU dürfen ungesichert preisgegeben werden, ohne Rückschlüsse auf sicherheitskritische Implementierungsdetails zuzulassen? Eine gestufte Freigabe der Graphtiefe abhängig vom Sicherheitslevel (analog zu `SecurityAccess-Leveln`) erscheint sinnvoll und sollte in einer Folgearbeit formal mit einem Informationsflussmodell untersucht werden.

Erweiterung auf Multi-ECU-Topologien. Die vorliegende Arbeit betrachtet den Signalgraphen einer einzelnen ECU. In modernen Fahrzeugarchitekturen mit zentralisierten Domänen-Steuergeräten und Zonal-Architekturen verteilen sich funktional zusammenhängende Signale jedoch über mehrere physische Steuergeräte und Gateways. Eine Erweiterung des Modells auf einen fahrzeugweiten, über Gateway-Routing aggregierten Signalgraphen $G_S^{\text{Fahrzeug}} = \bigcup_k G_S^{(k)}$ mit Cross-ECU-Abhängigkeitskanten sowie die Untersuchung, ob und wie die $\Theta(n^3)$ -Schranke bei verteilter, gatewaybasierter Vorberechnung pro ECU auf eine bessere Gesamtkomplexität reduziert werden kann, stellt eine der vielversprechendsten Richtungen für zukünftige Arbeiten dar.

Empirische Validierung an realen Fahrzeugdaten. Die in Abschnitt 5 gezeigten Laufzeitmessungen basieren auf einer Python-Referenzimplementierung mit synthetischen Zufallsgraphen. Eine Validierung mit realen ODX-Signalkatalogen heutiger Fahrzeuggenerationen (typischerweise mehrere Hundert bis wenige Tausend Signale pro Domäne) sowie ein Vergleich mit einer C/C++- oder Rust-Implementierung auf realer Diagnosehardware wären ein wichtiger nächster Schritt zur Beurteilung der Praxistauglichkeit, insbesondere hinsichtlich der konstanten Vorfaktoren, die in der asymptotischen Analyse naturgemäß unberücksichtigt bleiben.

Zusammengefasst zeigt diese Arbeit, dass die UDS-Signalvisualisierung weit über eine reine Implementierungsfrage hinausgeht: Sie lässt sich als formal beweisbares, graphentheoretisch fundiertes Optimierungsproblem fassen, dessen Lösung mittels des Subgraph Algorithmus sowohl beweisbar korrekt als auch unter realistischen Komplexitätsannahmen (SETH) asymptotisch optimal ist – ein Ergebnis, das die in der Originalarbeit zum Subgraph Algorithmus aufgezeigte Allgemeingültigkeit des Verfahrens für strukturelle Vergleichsprobleme eindrucksvoll für die Fahrzeugdiagnose bestätigt.

Literaturverzeichnis

- [1] International Organization for Standardization. *ISO 14229-1:2020 – Road vehicles – Unified diagnostic services (UDS) – Part 1: Application layer*. ISO, Genf, 2020.
- [2] International Organization for Standardization. *ISO 15765-2:2016 – Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) – Part 2: Transport protocol and network layer services*. ISO, Genf, 2016.
- [3] International Organization for Standardization. *ISO 13400-2:2019 – Road vehicles – Diagnostic communication over Internet Protocol (DoIP) – Part 2: Transport protocol and network layer services*. ISO, Genf, 2019.

- [4] International Organization for Standardization. *ISO 11898-1:2015 – Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*. ISO, Genf, 2015.
- [5] Epp, S. (2026). *The Subgraph Algorithm*. <https://github.com/hjstephan86/subgraph>.
- [6] Abboud, A., Backurs, A., Vassilevska Williams, V. (2015). *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. In: Proceedings of FOCS 2015, S. 59–78.
- [7] Hunkar, D. (2014). *Sequence Comparison: Algorithms and Applications*. Cambridge University Press.
- [8] ASAM e.V. *ASAM MCD-2D (ODX) – Open Diagnostic Data Exchange*, Version 2.2.0, 2017.
- [9] Impagliazzo, R., Paturi, R. (2001). *On the Complexity of k -SAT*. Journal of Computer and System Sciences, 62(2), S. 367–375.
- [10] Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2022). *Introduction to Algorithms*, 4. Auflage. MIT Press.